

AZURE DATA FACTORY

Definition --

Azure Data Factory (ADF) is a cloud-based data integration service that allows you to create workflows to move and transform data at scale.

Think of it as a **central dispatcher** for data. It doesn't store the data itself; instead, it provides a "factory" environment where you can build pipelines that reach into various sources (like an Excel file on your computer or a database in the cloud), grab the data, clean it up, and deliver it to a destination.

OR

Azure Data Factory is a **cloud ETL/ELT orchestration service** used to **ingest, move, and schedule data** between systems. It connects to many sources (databases, files, APIs) and coordinates pipelines, retries, and monitoring. ADF is great at **data movement and workflow control**, **not heavy transformations**.

Core Components

To understand how it works, think of it like a plumbing system:

- **Linked Services:** These are your "connection strings." They tell ADF *how* to connect to your data sources (e.g., your login credentials for a SQL Database).
- **Datasets:** These define the *structure* of the data (e.g., "I want to look at the 'Sales' table in that database").
- **Activities:** These are the *actions* performed on the data, such as copying it or running a script to transform it.
- **Pipelines:** This is the *workflow* that groups activities together.
- **Triggers:** These tell the pipeline *when* to run (e.g., every night at 2:00 AM).

A Real-World Example: Daily Sales Reporting

Imagine you run a retail chain. Your sales data is stored in **on-premises SQL databases** at each store location. However, your management team needs a **Power BI dashboard** that shows a consolidated view of all sales across the country.

The ADF Solution:

1. **Extract:** Every night, an ADF pipeline "wakes up" via a **Trigger**. It uses a **Linked Service** to securely connect to each store's local database.
2. **Transform:** The pipeline uses a **Mapping Data Flow** to clean the data—for instance, converting all local currencies into USD and removing duplicate entries.
3. **Load:** The cleaned data is then moved (the **Copy Activity**) into a central **Azure Synapse Analytics** warehouse.
4. **Visualize:** Once the data is loaded, ADF sends a signal to refresh the **Power BI** dashboard so the managers have fresh data by 8:00 AM.

ANOTHER SCENARIO

Scenario: Load daily sales data from an on-prem SQL Server into Azure Data Lake and prepare it for reporting.

How ADF works here:

1. **Extract:** ADF connects to SQL Server and reads the *Sales* table.
2. **Load:** ADF copies the data into **ADLS** (Bronze folder) as files.
3. **Orchestrate:** ADF triggers a **Databricks** job to clean/deduplicate data.
4. **Finish:** Clean data is written back to ADLS (Silver/Gold) and made ready for BI. (it's **Azure Databricks** that writes the transformed data back to ADLS (Silver/Gold).)

Key takeaway line (interview-ready):

"Azure Data Factory orchestrates data movement and pipeline workflows, while compute engines like Databricks handle complex transformations."

Why use it?

- **Code-Free:** You can build complex data movements using a drag-and-drop interface without writing a single line of Python or SQL (though you can if you want to).
- **Hybrid Power:** It bridges the gap between old "on-prem" servers and modern cloud storage.
- **Scalable:** Whether you're moving 10 rows or 10 billion rows, the service scales automatically to handle the load.

>> ETL PROCESS >>

For Extraction Process >>> ADF has all type of connector to Source to Destination

For Transformation Process >>> ADF has Data Transformation Functionality (Spark is running in background for Dataflow Functionality, it uses GUI but no code)

CREATING AZURE PORTAL ACCOUNT

>> Step 1

Creating resource group first -- **ADF_RG**

>> Step 2

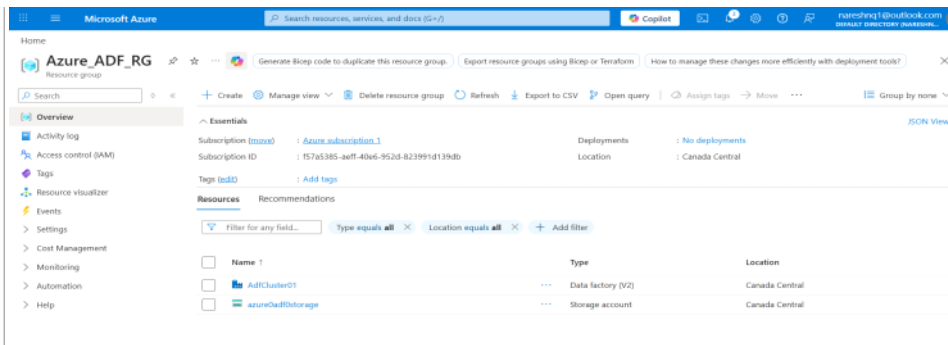
Two resources will be created under RG which are

Azure Data Factory and Azure Data Lake Storage

1. ADLS has been created [AdfCluster01](#)

While creating ADLS we have 4 types of Redundancy LRS,GRS,ZRS,GZRS
LRS is the cheapest one , keeps the cost low . Then GZRS

1. ADF has been Created -- [azure0adf0storage](#)

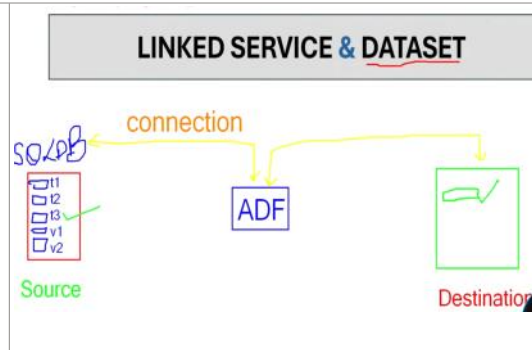


Azure Data Factory Fundamentals--

Core Components >>

To understand how it works, think of it like a plumbing system:

- **Linked Services:** These are your "connection strings." They tell ADF how to connect to your data sources (e.g., your login credentials for a SQL Database).
- **Datasets:** These define the *structure* of the data (e.g., "I want to look at the 'Sales' table in that database").
- **Activities:** These are the *actions* performed on the data, such as copying it or running a script to transform it.
- **Pipelines:** This is the *workflow* that groups activities together.
- **Triggers:** These tell the pipeline *when* to run (e.g., every night at 2:00 AM).



>> Step 3. Two Containers and folders in each container will be created under ADLS

Container1 name -- source (Here in this case ADF pulls the data from DB and land it in ADLS source folder)

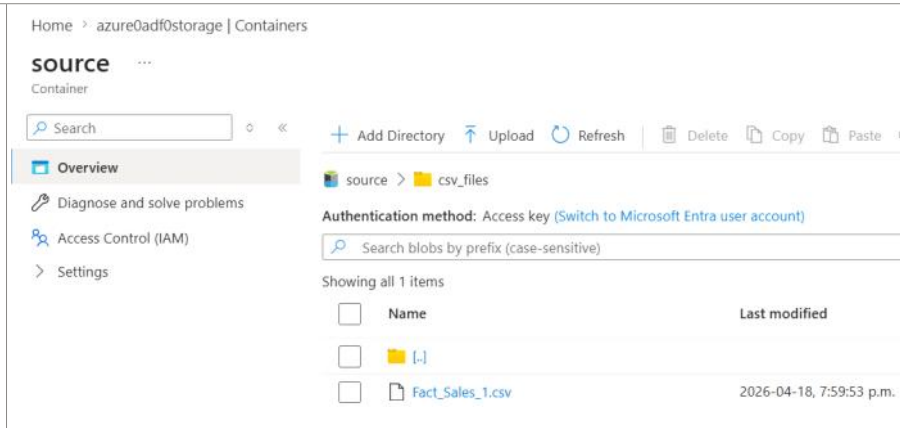
Two patterns exist:

1. Pull model (most common)
 - o ADF → pulls from DB → writes to ADLS ☒
2. Push/stream model (less common in beginner projects)
 - o System/tools → write to ADLS
 - o ADF → processes/orchestrates later

Source Container has been created :

>>

Later we will be creating a folder name under **Source** Container > **csv_files** > where we will bring the data from different sources , one file name **Fact_Sales_1 CSV** , will be directly uploading from Laptop



Container2 name -- destination created

Step4>> SETTING UP LINKED SERVICE USING ADF

LINKED SERVICE : IT sets up connection between ADF and ADLS , And can be found under Manage > Linked Service

LAUNCHING ADF STUDIO ---

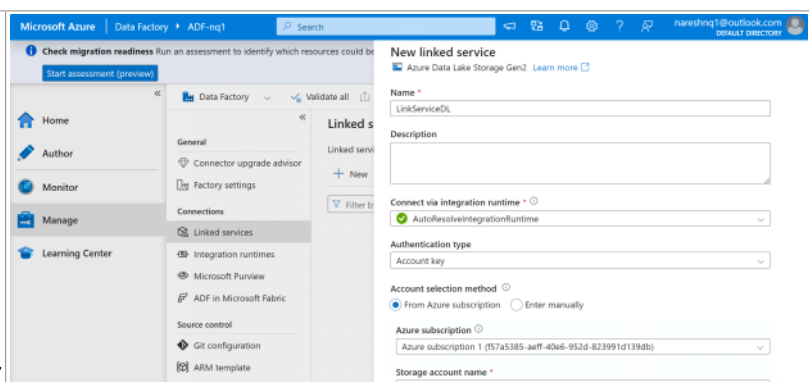
THEORY :

Typical ADF MENU Workflow (Interview Gold)

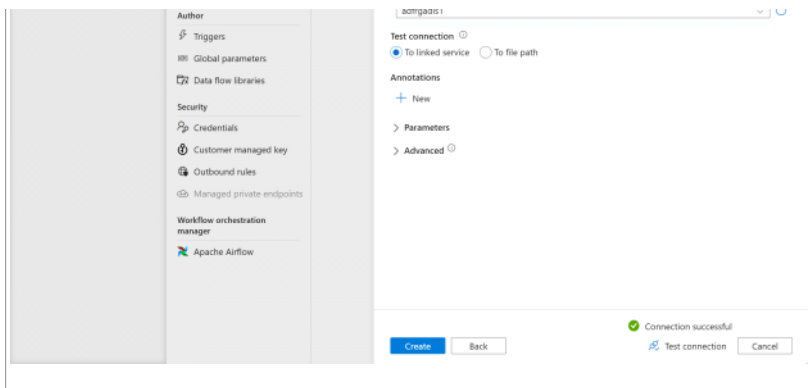
- 1 **Manage** – Set up linked services & IR
- 2 **Author** – Build pipelines & transformations
- 3 **Debug** – Test logic
- 4 **Publish** – Deploy changes
- 5 **Monitor** – Track execution & troubleshoot

One-Line Summary (Perfect to close answers)

"In ADF Studio, **Manage** sets up connections, **Author** builds pipelines, Monitor tracks execution, and Home provides navigation and quick access."



Linked Service has been created in ADF Named [LinkServiceDL](#)



NOW ADF IS LINKED WITH ADLS

NOTE : Before creating always make sure you test it .

CREATING 1 PIPELINE :

>>Pipeline1 using COPY ACTIVITY

Now we will be performing copy activity under AUTHOR TAB

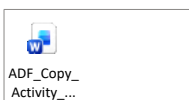
In First use case of copy activity this is what we will be doing >>moving data from data lake to data lake **ADLS (source) → ADF → ADLS (destination)**

Definition :

Copy Activity is used to extract data from different source systems and load it into a target system. It supports structured, semi-structured, and unstructured data.

>> We will be copying the data from source folder to destination folder using COPY ACTIVITY

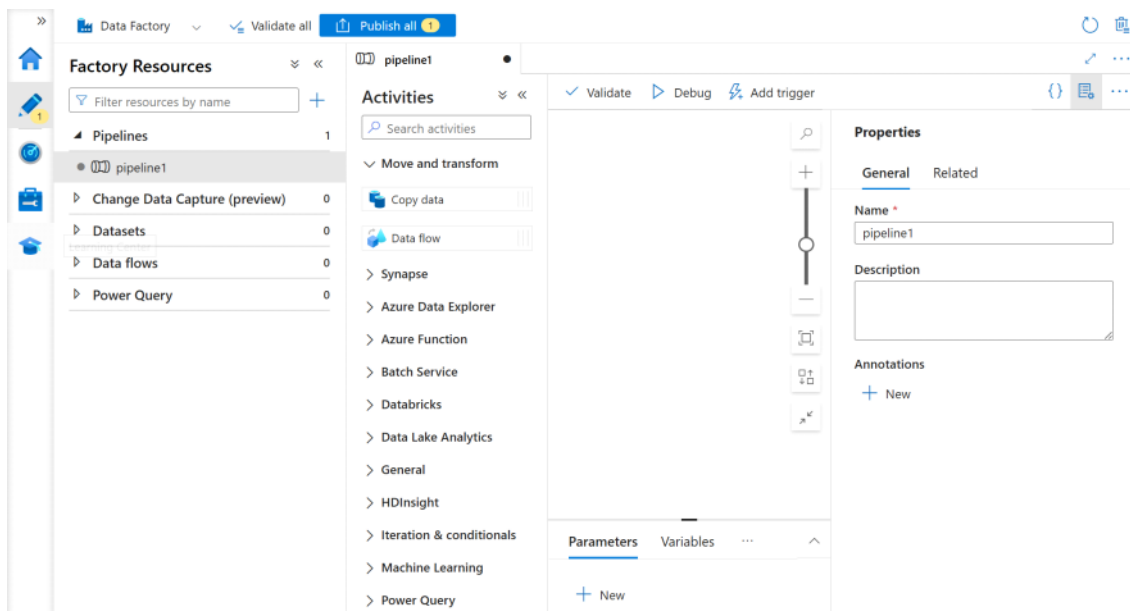
There are different use case of Copy Activity which are explained in word file below



AUTHOR TAB Description :

The screenshot shows the 'Factory Resources' section in the Azure Data Factory Author tab. The 'Pipelines' resource is highlighted, and the 'Validate all' button is visible at the top right.	Pipelines Pipelines orchestrate and control the execution of activities in Azure Data Factory. They define workflow logic, sequencing, dependencies, and scheduling. Change Data Capture (CDC) CDC captures only new or changed records from source systems. It helps implement efficient incremental loads instead of full refreshes. Datasets Datasets represent the structure and location of data used by activities. They act as metadata pointing to tables or files, not the data itself. Data Flows Data Flows provide a visual way to perform data transformations. They are used for joins, aggregations, and business logic at scale. Power Query Power Query enables low-code data preparation and transformation. It is commonly used for self-service or analyst-driven ingestion scenarios.
--	--

>> USE PIPELINE OPTION TO START (Naming as Pipeline1) and you will see Set of activities ---



Choosing Copy Data Activity From Activities --

Drag copy data into Canvas and fill the option required in Copy Data like General , Source , Sink , Mapping , Setting , User Properties

Copy Activity Components

- **Source:** Where data comes from (Azure SQL, ADLS, Blob, API, On-prem)
→ Table/file, query, or incremental (watermark)
- **Sink:** Where data goes (Azure SQL, Synapse, ADLS, Blob)
→ Insert/upsert, batch size, pre/post scripts
- **Mapping:** Column mapping
→ Auto, manual, schema drift

What we did so far >>

General tab >

we just named the activity as Copy CSV

Source Tab >

provide source after clicking +New then select data store as ADLS GEN 2, then select the type of data which is Delimited Text , then select linked Service that we created then select the dataset from source , **also set import schema as None** . Then OK .

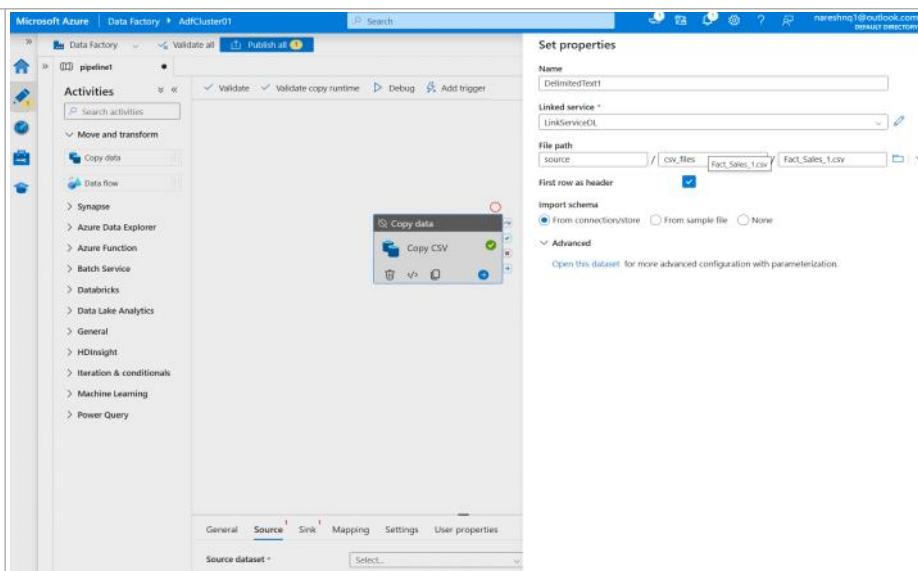
Sink Tab >

It is also known as Destination , Select +New , same steps as above , just added directory (csvfiles) as there was none in container

Run Pipeline > if we press Debug option it will migrate the data from source to destination folder with csvfiles

Check Storage to confirm ??

Yes Copy Activity has moved the data



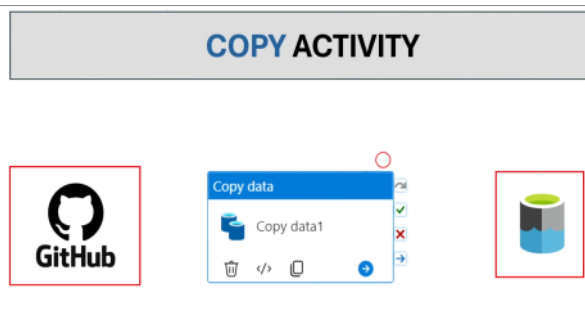
2nd USE CASE OF COPY ACTIVITY (From GitHub Repo to Destination Folder)

Here in First Case We moved the data from Data lake to data lake , but here it will be **From GitHub Repo to Destination Folder**

1:01:40

>> WE CAN ALSO USE EXTERNAL FILE TO RUN THE PIPELINE /PULLING THE DATA FROM EXTERNAL SOURCE LIKE API OR GITHUB And we have connectors for those as well , called **http connections**

From GitHub Repo to Destination Folder



we are using our GitHub profile file from there >>[Azure-Data-Factory-Project-I/Raw files/Fact_Sales_2.csv at main · Nareshnq/Azure-Data-Factory-Project-I](#)

Activity Name is **Copy Git**

NEW PIPELINE Name will be **pipeline2with_Git**

Same as before under Source Tab >

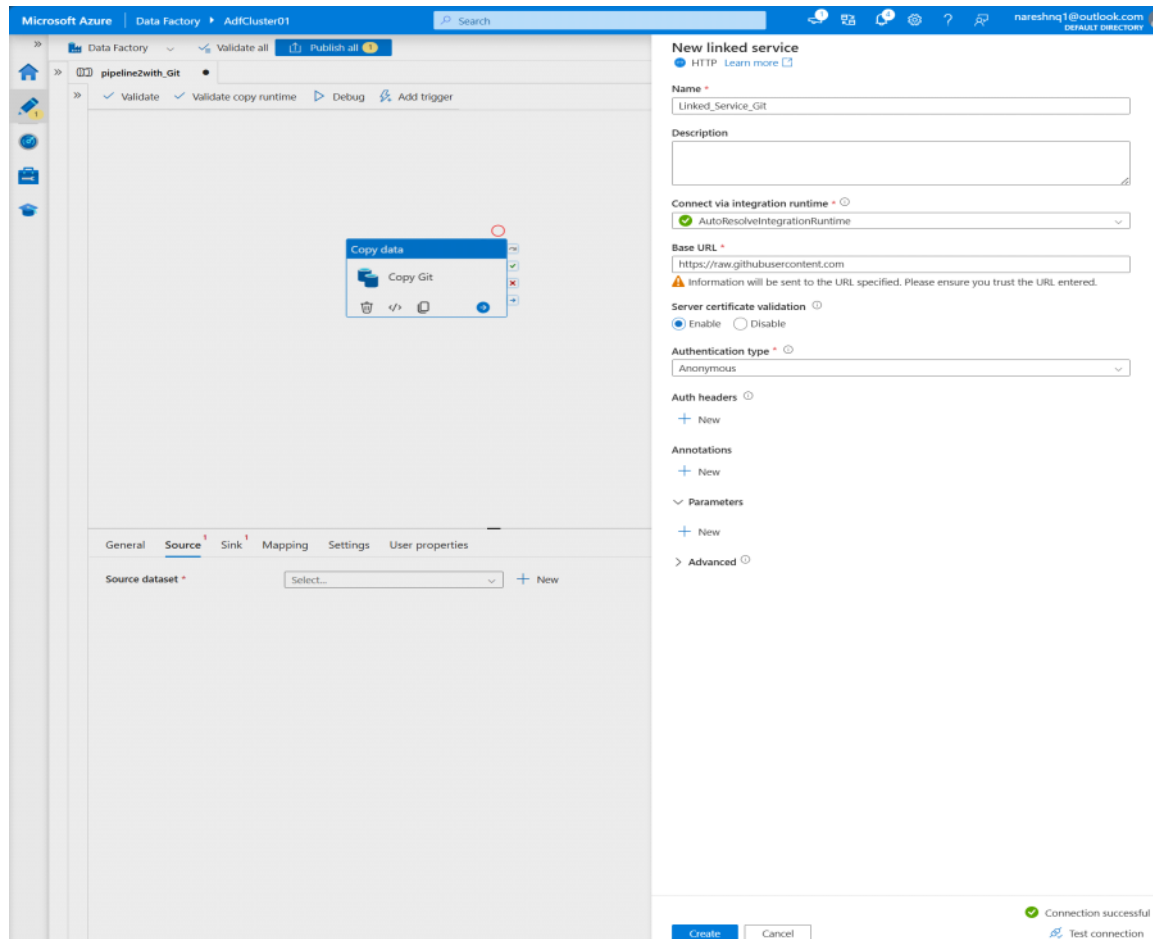
Click on +New >
Search HTTP as data store
(Recommended Approach for GitHub or any website /Rest API Can be used as well) >
DelimitedText > Set Properties Name = `source_git` > Linked Service > create new >
Name = `Linked_Service_Git`

Base URL : Go to GitHub Repo > find the file > click on Raw data option > it will give the webpage URL >

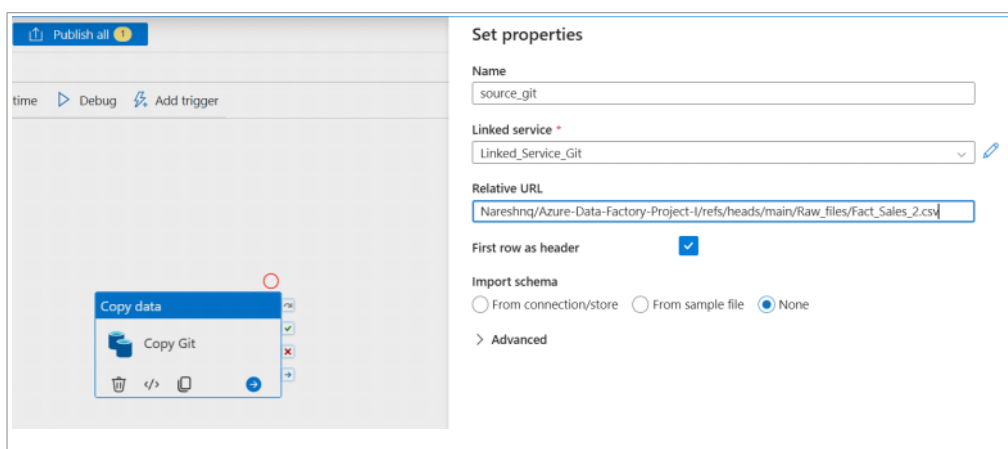
https://raw.githubusercontent.com/Nareshnq/Azure-Data-Factory-Project-1/refs/heads/main/Raw_files/Fact_Sales_2.csv is Complete URL
<https://raw.githubusercontent.com> is Base URL

[Nareshnq/Azure-Data-Factory-Project-1/refs/heads/main/Raw_files/Fact_Sales_2.csv](#) is Relative URL (We will need it as well)

Fill the Base URL > Authentication Type is Anonymous > Test Connection

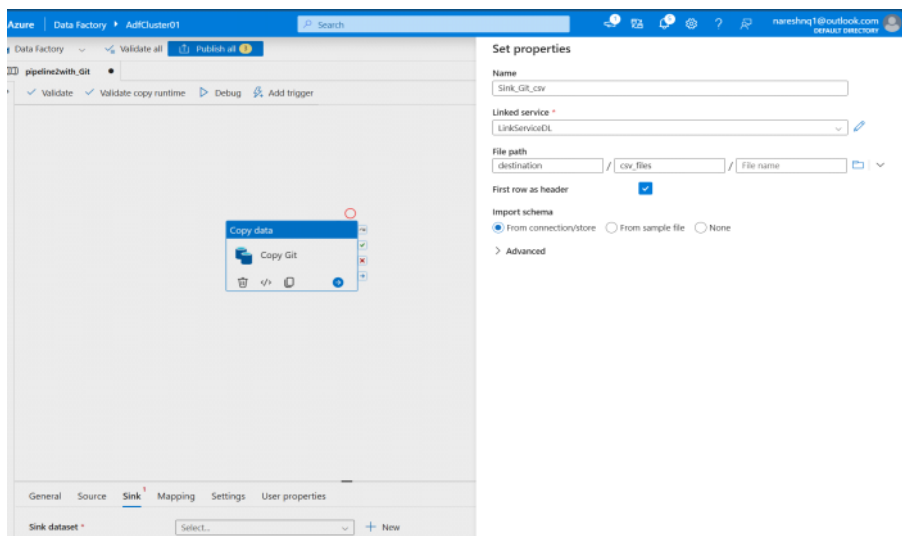


Now Create > fill the Relative URL now > click Ok



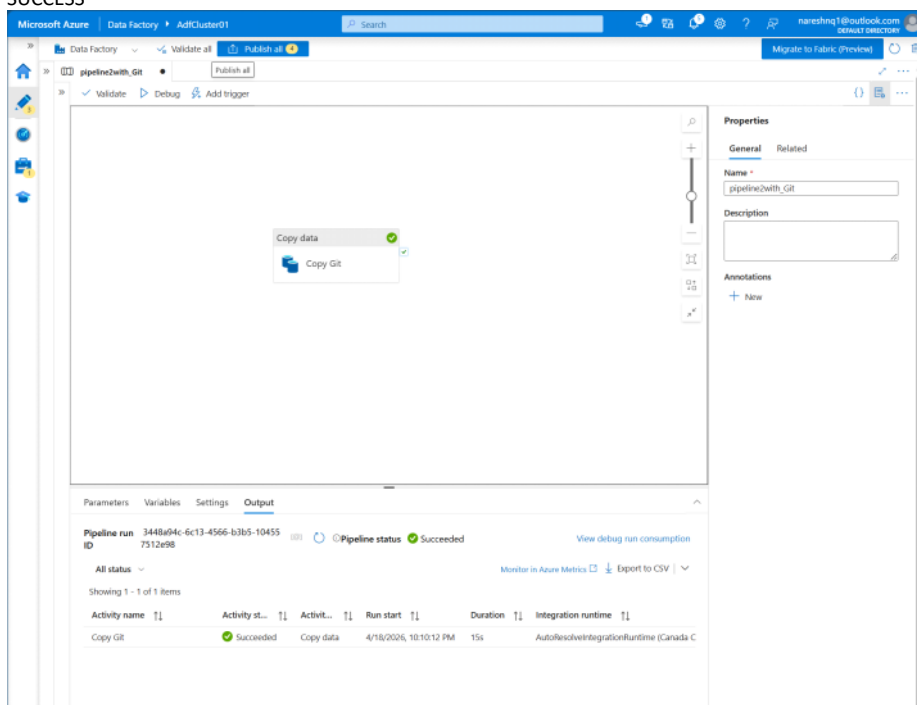
Source Tab is Done

Sink Tab > is same as before

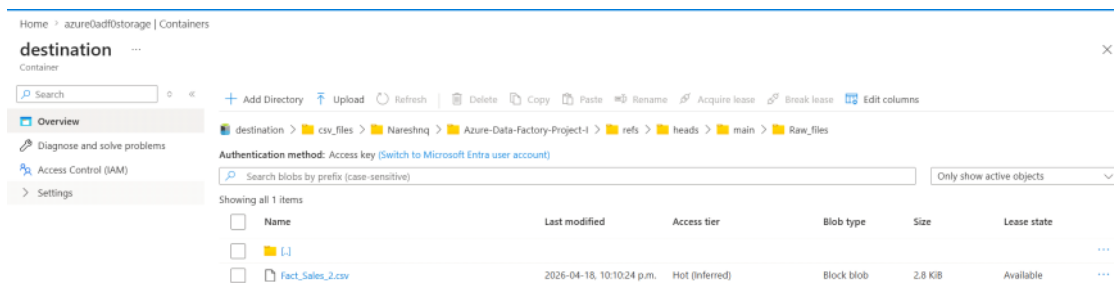


Click OK then
Click Debug

SUCCESS



It has successfully brought the file to destination folder but we see that file is in a new folder which is imported from GitHub . Because we did not mention the name while setting up destination (which is also recommended)



1:12:48

Metadata Activity in ADF:

The **Get Metadata** activity is used to **retrieve information *about* a file or folder** — not the data inside it, but properties like file name, size, last modified time, whether

Think of it as asking *"tell me about this file"* rather than *"give me the data in this file"*.

Suppose every morning a vendor drops a file called `sales_data.csv` into your ADLS folder. You want to copy it — but **only if it exists**. If it doesn't, you want to skip or log an error.

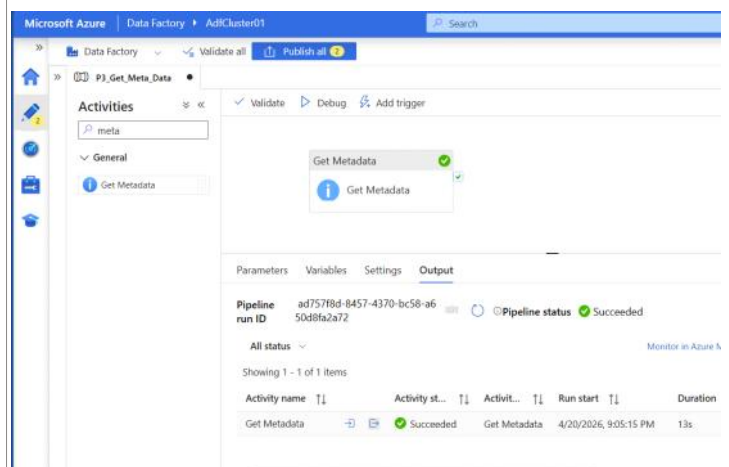
1. **Get Metadata activity** — points to sales_data.csv in ADLS and asks for the exists field property
2. **If Condition activity** — checks @activity('GetMeta').output.exists == true
3. **Copy activity (inside True branch)** — copies the file to destination
4. **Web/Fail activity (inside False branch)** — logs that file was missing

Metadata → (optional) ForEach → Action (like Copy)

1. **Metadata Activity**
→ gets list of files (child items)
2. **ForEach Activity**
→ loops through each file
3. Inside ForEach → **Copy Activity**
→ processes each file

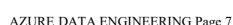
PIPELINE 3 - P3_Get_Meta_Data --

1. three files into destination folder >>>>>>>>>>>>>>>
2. checking meta data activity of those using folder csv_files



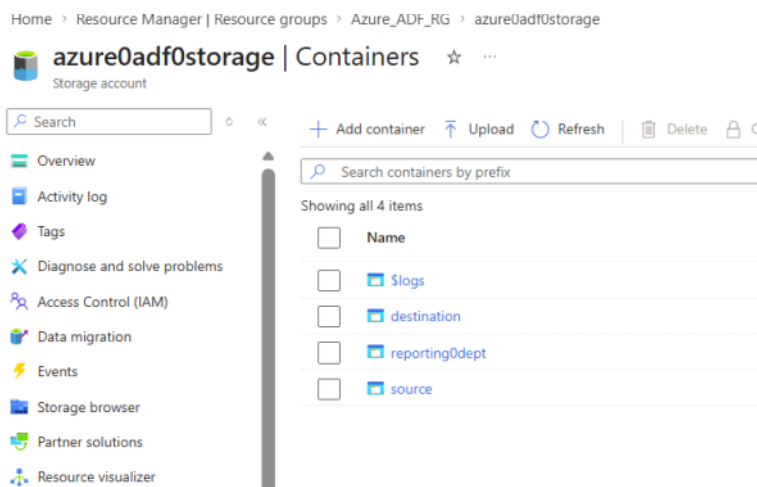
```
{
  "childItems": [
    {
      "name": "Bank_transaction_small_dataset.xlsx",
      "type": "File"
    },
    {
      "name": "Fact_Sales_1.csv",
      "type": "File"
    },
    {
      "name": "Fact_Sales_2.csv",
      "type": "File"
    },
    {
      "name": "Nareshnq",
      "type": "Folder"
    }
  ]
}
```

Concept : Copying data activity with a condition



Here we will be copying the data which will start with only Fact. Other will be skipped .

Lets create a new container named Reporting this is where we will be copying the files starts with fact



Step	Activity	Configuration
1	Get Metadata	Points to folder, fetches childItems
2	ForEach	Items = @activity('GetMetadata1').output.childItems
3 (inside ForEach)	If Condition	@startsWith(item().name, 'Fact') → True branch runs Copy

Concept Question : why do we need for loop here we already have if condition right ?

Answer :

They do completely different jobs

ForEach → **iterates** over the list, visiting each file one by one

If Condition → **evaluates** a true/false check on a single item

Think of it this way

Get Metadata returns 4 files

↓

ForEach picks up sales_jan.csv → If Condition → False → skip

ForEach picks up Fact_feb.csv → If Condition → True → Copy

ForEach picks up Fact_mar.csv → If Condition → True → Copy

ForEach picks up orders_apr.csv → If Condition → False → skip

Pipeline: pipeline4_Get_Meta_data_Adv

Step 1: Get Metadata Activity

- Added a **Get Metadata** activity as the first step in the pipeline
- This activity connects to a source dataset (**param_ds_source**) to retrieve metadata about files (e.g., list of child items / file names) from a storage location
- This gives you a list of files to iterate over

Step 2: ForEach Activity

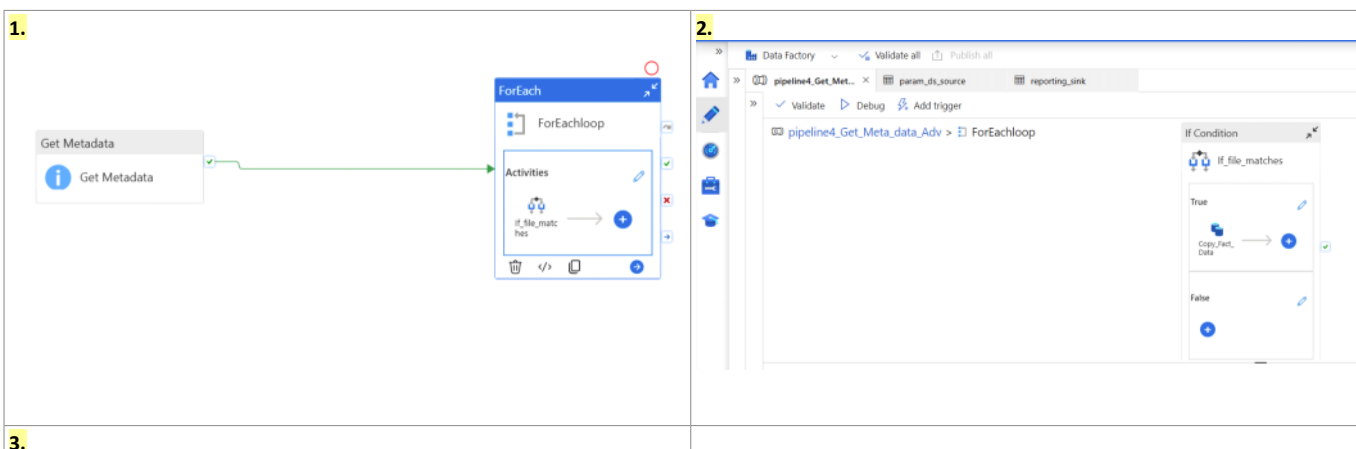
- Connected the **Get Metadata** output to a **ForEach** activity (named **ForEachloop**)
- The ForEach iterates over the list of files returned by Get Metadata
- Each iteration processes **one file at a time**
- Inside the ForEach, you added activities to handle each file

Step 3: If Condition Activity (inside ForEach)

- Inside the ForEach, added an **If Condition** activity named **IfFileMatches**
- This evaluates a condition for each file — checking if the current file name **matches a specific pattern or criteria**
- This acts as a filter so only relevant files are processed

Step 4: Copy Data Activity (inside If Condition — True Branch)

- Inside the **True branch** of the If Condition, added a **Copy Data** activity named **Copy_Fact_Data**
- This copies the matched file from the source (**param_ds_source**) to the sink (**reporting_sink**)
- The **False branch** was left empty — meaning non-matching files are simply skipped with no action





Result :

The screenshot shows the Azure Data Factory pipeline run output. The pipeline run ID is 42005b53-40d2-4f1d-a628-b0e5a71e4f8c, and the pipeline status is 'Succeeded'. The output table shows the following activities:

Activity name	Activity status	Activity type	Run start	Duration	Integration runtime	User properties	Activity ID
If_file_matches	Succeeded	If Condition	4/21/2026, 12:00:37 AM	Less than 1s			1bd78b8
Copy_Fact_Data	Succeeded	Copy data	4/21/2026, 12:00:22 AM	13s	AutoResolveIntegrationRuntime (Canada Central)		302153f8
If_file_matches	Succeeded	If Condition	4/21/2026, 12:00:22 AM	15s			838236e
Copy_Fact_Data	Succeeded	Copy data	4/20/2026, 11:59:55 PM	20s	AutoResolveIntegrationRuntime (Canada Central)		1387e3ff
If_file_matches	Succeeded	If Condition	4/20/2026, 11:59:54 PM	27s			9c66cb1e
If_file_matches	Succeeded	If Condition	4/20/2026, 11:59:53 PM	Less than 1s			aa107e7f
ForEachloop	Succeeded	ForEach	4/20/2026, 11:59:52 PM	48s			aecefc9f
Get Metadata	Succeeded	Get Metadata	4/20/2026, 11:59:38 PM	13s	AutoResolveIntegrationRuntime (Canada Central)		8d8e9fe7

I have added bank_txns1 and 2 , I will move them to bank0dept using above method

pipeline4_1-

Successfully deployed ,

DATA FLOW ACTIVITY :

How Data Flow Works

Outside the ForEach (after all files are copied)

- Let ForEach finish copying **all matching files** first
- Then run Data Flow **once** on the entire landed dataset
- More efficient if Data Flow can handle all files together

Key Difference — Copy vs Data Flow

	Copy Activity	Data Flow Activity
Purpose	Move raw data as-is	Transform / clean data
Transformations	None	Full ETL logic
Performance	Very fast	Runs on Spark cluster
Use case	Land raw files	Business logic layer

Important Thing to Know

- Data Flow runs on a **Spark cluster behind the scenes** (similar to what you've been learning!)
- It spins up an **Integration Runtime** (cluster) when triggered — so first run has a ~1-2 min warm-up
- You can use **debug mode** to preview transformations row by row

Let's say 3 files landed in reporting_sink:
reporting_sink/

```
├── sales_jan.csv
├── sales_feb.csv
└── sales_march.csv
```

How Data Flow Reads Them

When Data Flow source points to the **folder** (not a specific file),

it reads **all 3 files at once as one combined dataset**

— like this:

sales_jan.csv →

sales_feb.csv → combined into ONE data frame →

transform → sink

sales_march.csv →

It behaves exactly like **Spark reading a folder**

— which you already know! Under the hood it does:

python

```
df = spark.read.csv("reporting_sink/") # reads all files in folder
```

Simple Example

3 files content:

sales_jan.csv sales_feb.csv sales_march.csv

```
Jan, 1000    Feb, 2000    Mar, 1500
Jan, 800    Feb, 1200    Mar, 900
```

Data Flow combines them first:

```
Month | Amount
Jan   | 1000
Jan   | 800
Feb   | 2000
Feb   | 1200
Mar   | 1500
Mar   | 900
```

Then applies your transformation once (e.g., total by month):

```
Month | Total
Jan   | 1800
Feb   | 3200
Mar   | 2400
```

One Thing to Be Careful About

If your 3 files have **different schemas** (different columns), Data Flow may throw an error or misalign columns. So ideally all landed files should have the **same structure** — which in your case they likely do since they all passed the If_file_matches condition ☒

We have added new activity called data flow activity to pipeline 4, we have made the clone of pipeline 4. Then we will be performing different transformation under this data flow activity. At the end we will connect it to sink (choosing destination folder). There it will be landed with transformation. After validating this pipeline

we will be setting up scheduled trigger as well to see the working of it .

pipeline4_Get_Met...

dataflow1

Validate

Data flow debug

Debug Settings

source1

select1

filter1

VISA

Master

amex

derivedColumn

Derived column's settings

Optimize

Inspect

Data preview

Number of rows

INSERT 100

UPDATE 0

DELETE 0

UPSERT 0

LOOKUP 0

ERROR 0

TOTAL 343

Refresh

Typecast

Modify

Map drifted

Statistics

Remove

Export to CSV

transaction_id	transactional_date	product_id	customer_id	payment	price
6	2021-05-04 06:58:00.000	P0385	6	americanexpress	14.69

Cancel

Properties

General

Related (1)

Name *

dataflow1

Description

Group by :

pipeline4_Get_Met...

dataflow1

Validate

Data flow debug

Debug Settings

source1

select1

filter1

VISA

Master

amex

aggregate1

derivedColumn

Aggregate settings

Optimize

Inspect

Data preview

Number of rows

INSERT 13

UPDATE 0

DELETE 0

UPSERT 0

LOOKUP 0

ERROR 0

TOTAL 13

Refresh

Typecast

Modify

Map drifted

Statistics

Remove

Export to CSV

customer_id	product_id
-1	P0022
0	P0314
1	P0179
2	P0640
3	P0746

Cancel

Properties

General

Related (1)

Name *

dataflow1

Description

Data Factory | Validate all | Publish all | Migrate to Fabric (Preview)

pipeline4_Get_Met... | dataflow1

Validate | Data flow debug | Debug Settings

Properties

General | Related (1)

Name *
dataflow1

Description

Cancel

Alter row settings | Optimize | Inspect | Data preview

Output stream name *
alterRow1 | Learn more

Description
Add expressions to alter rows | Reset

Incoming stream *
aggregate1

Alter row conditions *

With Trigger scheduled

Microsoft Azure | Data Factory | AdfCluster01

Home | Author | Monitor | Manage | Learning Center

Dashboards | Runs | Pipeline runs | Trigger runs | Change Data Capture (previ...) | Runtimes & sessions | Integration runtimes | Data flow debug | Notifications | Alerts & metrics

All pipeline runs > pipeline4_Get_Meta_data_Adv_copy1 - Activity runs

Rerun | Cancel | Refresh | Update pipeline | List | Gantt

Activity runs

Pipeline run ID: 8d404bd6-1399-4738-96a4-3de5336a8172

All status | List | Monitor in Azure Metrics | Export to CSV

Showing 1 - 11 items

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime	User prop...	Activity run
Get Metadata	Succeeded	Get Metadata	4/21/2026, 5:50:04 PM	13s	AutoResolveIntegrationRuntime (Canada Central)		ca1982cc-60
ForEachloop	Succeeded	ForEach	4/21/2026, 5:50:17 PM	52s			e0b262b6-2
If_file_matches	Succeeded	If Condition	4/21/2026, 5:50:18 PM	Less than 1s			502de0d0-0
If_file_matches	Succeeded	If Condition	4/21/2026, 5:50:19 PM	Less than 1s			6b6ae34a-3
If_file_matches	Succeeded	If Condition	4/21/2026, 5:50:20 PM	Less than 1s			b32814ba-4
If_file_matches	Succeeded	If Condition	4/21/2026, 5:50:21 PM	21s			8ca9c02c-98
Copy_Fact_Data	Succeeded	Copy data	4/21/2026, 5:50:22 PM	19s	AutoResolveIntegrationRuntime (Canada Central)		06934584-4
If_file_matches	Succeeded	If Condition	4/21/2026, 5:50:42 PM	24s			2bbfc3f2-8d
Copy_Fact_Data	Succeeded	Copy data	4/21/2026, 5:50:43 PM	23s	AutoResolveIntegrationRuntime (Canada Central)		763c0e59-a7
If_file_matches	Succeeded	If Condition	4/21/2026, 5:51:07 PM	Less than 1s			1d398955-br
Data flow_Transformation	Succeeded	Data flow	4/21/2026, 5:51:09 PM	3m 24s	AutoResolveIntegrationRuntime (Canada Central)		58585b5b-e

So far everything worked . New Folder has been created with transformed data .

Home > Resource Manager | Resource groups > Azure_ADF_RG > azure0adf0storage | Containers

reporting0dept

Container

Search

+ Add Directory | Upload | Refresh | Delete | Copy | Paste | Rename | Acquire lease | Break lease | Edit columns

Overview

Diagnose and solve problems | Access Control (IAM) | Settings

reporting0dept

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Showing all 2 items

Name	Last modified	Access tier
csv_files	2026-04-21, 12:00:11 a.m.	
dataflowsOutput	2026-04-21, 5:54:26 p.m.	

Microsoft Azure

Data Factory > AdfCluster01

Search

Home

Author

Monitor

Manage

Learning Center

Dashboards

Runs

Pipeline runs

Trigger runs

Change Data Capture (previ...

Runtimes & sessions

Integration runtimes

Data flow debug

Notifications

Alerts & metrics

Pipeline runs

Triggered

Debug

Retrun

Cancel options

Refresh

Edit columns

List

Gantt

Filter by run ID or name

Eastern Time (US & C... : Last 24 hours

Pipeline name : All

Status : All

Runs : Latest runs

Copy

Triggered by : All

Add filter

Showing 1 - 2 items

☐	Pipeline name ↑↓	Run start ↑↓	Run end ↑↓	Duration	Triggered by	Status ↑↓
☐	pipeline4_Get_Meta_data_Ad...	4/21/2026, 5:56:00 PM	4/21/2026, 6:01:11 PM	5m 12s	trigger1_selected_files	Succeeded
☐	pipeline4_Get_Meta_data_Ad...	4/21/2026, 5:50:01 PM	4/21/2026, 5:54:34 PM	4m 34s	trigger1_selected_files	Succeeded

=====